

Efficient Memory Management for Large Language Model Serving with PagedAttention

1 Paper summary

This paper looks at the problem of serving large language models efficiently. The main point is that performance is not only limited by computation, but also by how GPU memory is used, especially for the KV cache during generation.

The KV cache is tricky to manage because it grows dynamically and is usually stored in contiguous memory. This leads to fragmentation and wasted space, which in turn limits how many requests can be handled at the same time. So even if the GPU has enough compute, memory becomes the bottleneck.

To solve this, the authors propose PagedAttention. The idea is inspired by virtual memory in operating systems: instead of storing the KV cache in contiguous chunks, it is divided into fixed-size blocks that can be allocated and placed anywhere. This makes memory usage more flexible and reduces fragmentation, even if it adds some complexity.

On top of this, they build vLLM, which is a serving system that uses this block-based approach. It also includes scheduling and sharing mechanisms, so that different requests (or different outputs of the same request) can reuse parts of the KV cache instead of duplicating it. This is especially useful for things like beam search or parallel sampling.

The results show that vLLM significantly improves throughput (around 2–4x compared to other systems) while keeping similar latency. It also reduces memory waste almost completely. So the main contribution is not a new model, but a better way to manage memory for serving.

2 Contributions

The first contribution is identifying KV-cache memory management as a key bottleneck in LLM serving. The paper shows that a lot of memory is wasted due to fragmentation and over-allocation, which limits performance more than expected.

The second contribution is PagedAttention. By using fixed-size blocks instead of contiguous memory, it reduces fragmentation and allows more flexible allocation. This is similar to how paging works in operating systems, even if applying it here is not completely trivial.

The third contribution is the vLLM system built on top of it. It includes scheduling and block-sharing mechanisms that improve efficiency, especially for more complex decoding methods. The evaluation shows clear improvements over existing systems, even if the gains depend on the workload.

3 Problems

One limitation is that the benefits depend a lot on the type of workload. The system works best with long sequences or decoding methods like beam search, where there is more opportunity for sharing KV cache blocks. For simpler cases, the improvement might be smaller.

Another issue is that the design is more complex than traditional approaches. Managing blocks, mappings, and reference counts introduces extra overhead and makes the system harder to implement and debug. Even if the results are good, this added complexity could be a downside in practice.

A third point is that the paper focuses mainly on memory efficiency inside the GPU, but doesn't explore other possible bottlenecks in real deployments. For example, things

like network overhead, multi-tenant environments, or integration with different systems are not really discussed. So while the results are convincing, there are still some open questions for large-scale usage.